

STOCK

PROJECT TYPE

A secure stock-management and trading simulation for an administrator. It does not use a real stock exchange; its live price movement is simulated in the backend.

Angular + Spring Boot + Oracle Database

architecture, technology stack, database design, security,
business logic, live pricing, APIs, testing, and presentation tips.

Prepared for demonstration, viva, and project presentation

Project overview

What the project does, who uses it, and why it matters.

The problem

Managing stocks, customers, customer holdings, and transaction records manually can lead to inconsistent inventory, missing audit history, and incorrect portfolio values. A web application centralizes these activities and applies rules before data is saved.

The solution

Stock Manager is a full-stack web application for an administrator. It manages the stock inventory and customer records, executes buy and sell transactions, updates customer portfolios, saves transaction history, and provides reports.

Primary user

- The administrator logs in and uses the protected application pages.
- The administrator can create, search, update, or delete stocks and customers.
- The administrator can buy stock for a customer or sell stock already owned by that customer.
- The administrator can inspect portfolio value, transaction history, and reports.

Key result

BUSINESS VALUE

The application keeps inventory, holdings, transaction records, and current market price aligned. This gives one reliable source of truth instead of separate manual records.

Main functional features

A summary of the modules that make up the application.

Dashboard

Shows headline metrics such as listed stocks, customers, transactions, portfolio value, and inventory information.

Stocks

Supports stock CRUD operations, search, pagination, current price, available quantity, and live movement color.

Customers

Supports customer CRUD operations, search, form validation, and customer portfolio access.

Trade

Executes buy and sell orders with business-rule checks before database changes are committed.

Transactions

Shows a complete history of buys and sells, including the execution price and date/time.

Portfolios and reports

Displays current customer holdings and reports such as most traded stock, highest price, and total asset value.

Optional enhancement

An optional AI assistant is available through the Groq API. It can answer questions about the workspace data and prepare a trade proposal for confirmation. It does not execute a trade by itself.

Technologies used

Each technology has a specific responsibility in the project.

Frontend - Angular 22

Angular provides the browser application. TypeScript is used for components, routes, typed models, API services, and application logic.

Reactive frontend tools

Angular Signals manage local UI state. RxJS manages asynchronous API requests and the one-second stock-price refresh. Reactive Forms validate user input.

Backend - Java 17 and Spring Boot 3

Spring Boot provides the REST API, dependency injection, configuration, validation, logging, scheduling, and application startup.

Persistence - Spring Data JPA and Hibernate

JPA maps Java entities to Oracle tables. Repositories perform database operations while services keep the business logic separate.

Database - Oracle

Oracle stores persistent stock, customer, portfolio, transaction, and administrator data. The schema is defined in sqlscript.sql.

Security and tools

Spring Security, BCrypt, HTTP sessions, CSRF protection, CORS, Maven, npm, Angular CLI, and Swagger/OpenAPI are also used.

System architecture

The design follows a simple and maintainable three-tier structure.

Three tiers

1. Angular frontend

The browser UI contains pages, forms, route guards, validation messages, loading states, responsive styling, and API calls.

2. Spring Boot backend

The backend exposes REST endpoints, authenticates the user, validates requests, applies business rules, and returns JSON responses.

3. Oracle database

Oracle persists the application data. Hibernate and Spring Data JPA translate repository operations into database queries.

Layered backend pattern

CONTROLLER -> SERVICE -> REPOSITORY -> DATABASE

Controllers receive HTTP requests. Services contain business rules. Repositories access Oracle through JPA. This separation makes the project easier to test, understand, and maintain.

Example request flow

- The user submits a trade on the Angular Trade page.
- Angular sends a JSON request to a protected Spring Boot REST endpoint.
- The service checks stock availability or customer ownership.
- Repositories update Oracle tables inside a database transaction.
- The backend returns JSON and Angular updates the visible page.

Oracle data model

The schema separates current state from historical transaction records.

Core tables

STOCK

Stores stock symbol, company name, CURRENT_PRICE, and AVAILABLE_QUANTITY. CURRENT_PRICE is the current simulated market price.

CUSTOMER

Stores the customer ID, customer name, and email address.

STOCK_TRANSACTION

Stores every buy or sell action with customer, stock, transaction type, quantity, trade price, and transaction date.

CUSTOMER_PORTFOLIO

Stores each customer holding: customer ID, stock symbol, and the number of shares currently owned.

ADMIN_CREDENTIAL

Stores the administrator username and BCrypt password hash. Plain-text passwords are not stored.

Relationships

- One customer can have many transactions and many portfolio holdings.
- One stock can appear in many transactions and in many customer portfolios.
- The portfolio uses customer ID plus stock symbol to identify one customer-stock holding.
- Transaction price is historical; current price is stored separately in STOCK.CURRENT_PRICE.

Spring Boot backend

REST APIs, business rules, transactions, and persistence.

Important backend components

Controllers

StockController, CustomerController, TransactionController, PortfolioController, ReportController, AuthController, and AssistantController expose REST endpoints.

Services

StockService, CustomerService, TransactionService, PortfolioService, ReportService, PriceSimulationService, and AssistantService contain application logic.

Repositories

Spring Data JPA repositories retrieve and persist entities. The project also uses locking queries when a stock row must be updated safely.

Entities and DTOs

Entities map to database tables. DTOs define clean request and response shapes for the API.

Validation and errors

- Bean Validation checks required values, valid prices, quantities, symbols, names, and email addresses.
- A global exception handler converts failures into consistent API error responses.
- Application logging records meaningful events such as stock creation and completed trades.

How buying and selling works

The most important part of the project is maintaining correct data during a trade.

Buy stock flow

- Find the customer and lock the selected stock for update.
- Verify that the requested quantity does not exceed available stock quantity.
- Reduce STOCK.AVAILABLE_QUANTITY.
- Create or update the customer portfolio holding.
- Save a BUY transaction using the latest STOCK.CURRENT_PRICE.

Sell stock flow

- Find the customer, lock the stock, and find the customer holding.
- Verify that the customer owns at least the requested quantity.
- Reduce or delete the portfolio holding if all shares are sold.
- Increase STOCK.AVAILABLE_QUANTITY.
- Save a SELL transaction using the latest STOCK.CURRENT_PRICE.

Why database transactions matter

ATOMIC OPERATION

A trade changes multiple records. The database transaction ensures all required changes succeed together, or all changes are rolled back if an error occurs.

Simulated live market feature

The project now has natural price movement without a third-party market-data API.

Backend simulation

- Spring scheduling is activated with `@EnableScheduling`.
- `PriceSimulationService` runs every 1,000 milliseconds.
- It locks stock rows and changes each `CURRENT_PRICE` by a small random amount, roughly +/- 0.15 percent.
- Prices are rounded to two decimal places and never allowed to go below 0.01.
- Only `CURRENT_PRICE` is changed. No previous-price or price-history table is stored.

Frontend display

- The Stocks page refreshes the stock list every second using RxJS interval and `switchMap`.
- Angular compares each new price with the previous displayed price.
- Price text turns green when the value increases and red when it decreases.
- The Trade page also refreshes stock data every second, so its market price stays current.

Why backend pricing is better

The price changes in the database, not only in the browser. Therefore the trade service, the portfolio calculations, and the screen all use the same current value. A real market-data provider could replace the simulator later.

Angular user experience

The frontend is designed as a responsive single-page application.

Angular implementation

- Lazy-loaded Angular routes improve loading by requesting a page component only when it is needed.
- Route guards keep protected pages unavailable until the administrator is authenticated.
- Angular Signals hold state such as stocks, customers, loading, errors, form state, and price movement direction.
- Reactive Forms provide field validation before create, update, and trade submissions.
- HttpClient and an API service call the Spring Boot backend and handle error responses.

Usability features

- Search and pagination make stock and customer data easier to browse.
- Loading, error, success, and empty states communicate what is happening to the user.
- The interface includes dashboard, stocks, customers, trade, transactions, portfolios, reports, login, and not-found pages.

Application security

Security is implemented on both the backend and browser boundary.

Authentication

- Spring Security protects API endpoints. The login endpoint is the public entry point.
- Passwords are stored using BCrypt hashes rather than plain text.
- After login, the application uses an HTTP session cookie.
- Session fixation protection changes the session ID after successful login.

Request protection

- CSRF protection is enabled using a token cookie.
- CORS is configured so the Angular development origin can call the backend with credentials.
- The session cookie uses HttpOnly and SameSite settings to reduce common browser risks.
- Database passwords and optional AI API keys are kept in backend/.env, which is ignored by Git.

Important presentation point

For a local training project, the README shows an Oracle SYSTEM setup. In a production system, the application should use a dedicated database user with least-privilege permissions.

Quality and delivery

How the application is documented, verified, and demonstrated.

Representative REST endpoints

- GET and POST /api/stocks - list or create stocks.
- GET, PUT, and DELETE /api/stocks/{symbol} - read, update, or delete one stock.
- GET and POST /api/customers - list or create customers.
- POST /api/transactions/buy and POST /api/transactions/sell - execute trades.
- GET /api/portfolios and GET /api/reports/* - portfolio and reporting information.

Documentation and checks

- Swagger/OpenAPI provides interactive API documentation at the backend Swagger UI route.
- Backend automated checks run with: mvnw.cmd test.
- Frontend production verification runs with: npm run build.
- The application is developed locally with Oracle on port 1521, Spring Boot on port 8080, and Angular on port 4200.

Suggested demo order

- Log in, open the dashboard, create or view a stock, and show the live price changing.
- Open the Trade page, buy shares for a customer, then show the transaction and portfolio changes.
- Open Reports and explain the management information available to the administrator.

How to explain the project

Use this concise script during a demo or viva.

One-minute project explanation

My project is a full-stack Stock Manager built using Angular, Spring Boot, and Oracle Database. It allows an administrator to manage stocks and customers, perform buy and sell transactions, track customer portfolios, and view reports. Angular provides the responsive frontend, while Spring Boot exposes secure REST APIs and uses Spring Data JPA to connect with Oracle. Trade operations use validation, transactions, and database locking to keep inventory and portfolio data consistent. I also added simulated live stock prices: Spring Boot updates `STOCK.CURRENT_PRICE` every second, Angular refreshes the price every second, and the Stocks page shows green for an increase and red for a decrease. Security uses Spring Security, BCrypt password hashing, sessions, CSRF protection, and protected routes.

Useful viva answers

Why use a transaction?

One buy or sell operation updates inventory, portfolio, and transaction history. A database transaction prevents partial updates.

Why save transaction price?

`CURRENT_PRICE` can change every second. The saved transaction price preserves the exact price used at the time of the trade.

Why use JPA?

JPA maps Java entities to Oracle tables and reduces repetitive database access code while repositories keep persistence logic organized.

How would you add real data?

Replace the simulated price calculation with a backend service that calls a market-data provider. The frontend and trade workflow can remain the same.